

Commandes git de base

Configurer git

```
git config user.name
```

```
git config user.email
```

Commandes de base

Tirer un dépôt existant

```
git clone
```

Initialiser un nouveau dépôt

```
git init
```

Se positionner dans l'arbre de versions

```
git checkout
```

On peut choisir une branche, un tag, voire un hash de commit.

Etat du dépôt / modifications locales

```
git status
```

Voir l'historique du dépôt

```
gitk --all &
```

Lance une fenêtre graphique, pas très jolie, mais qui rend le service.

```
git log
```

Permet de visualiser une information d'historique dans la console, mais on ne voit pas les branches qui partent de droite et de gauche.

```
git log -n 1
```

Permet de visualiser rapidement les détails du commit sur lequel on pointe actuellement.

Noter que chaque commit est identifié par un hash, en l'occurrence un hash SHA1.

 **Astuce : nombre de caractères d'un hash de commit**

Saisir les X premiers caractères d'un hash de commit suffit.

Par exemple, les deux commandes suivantes sont équivalentes :

```
git checkout 1c0f364
git checkout 1c0f3644e0329113608db853e8a3aa3468e99648
```

Les plateformes git (telles que GitHub) utilisent régulièrement les 7 premiers caractères uniquement

Raccourcir le nombre de caractères peut avoir son intérêt pour la lisibilité, notamment dans les commit logs et les textes des issues.

Check commit vient également avec :

- un auteur, identifié par un nom et un email,
- une date,
- un texte descriptif, qu'on appelle le *commit log*.

Dans gitk, on peut également voir rapidement les modifications apportées par chaque commit.

Créer un commit

`git commit` Mais pas facile à utiliser en ligne de commandes.

Plus facile d'utiliser l'interface graphique, pas très jolie, mais fournie de base avec Git Bash. `git gui &`

Branches

`git branch` ou `git branch -vv`

Permet de lister les branches locales dans le dépôts (avec plus ou moins de détails).

`git branch -a` ou `git branch -avv`

Permet de lister les branches locales et distantes (avec plus ou moins de détails encore une fois).

Pour tirer une nouvelle branche, personnellement je préfère utiliser la séquence suivante :

`git checkout ...` => d'abord je sélectionne la d'où je veux repartir.

`git checkout -b ...` => permet de tirer la nouvelle branche et de l'utiliser directement.

Tags

`git tag` Permet de visualiser les tags.

`git tag V1.0.3` Pose le tag "V1.0.3".

Comparer des versions du repos

`git diff`

`git diff --stat`

Pousser / récupérer des commits entre repos

git fetch Récupère une mise à jour de l'historique depuis le dépôt distant.

git push Pousse les commits locaux vers le dépôt distant.

Upstream branch **git push --set-upstream origin <local-branch>** Tip : Je laisse git me rappeler la commande, que je peux simplement copier / coller.

git push --tags Par défaut, git ne pousse pas les tags

Dangers du **git push --force**

Commit ligne à ligne avec **git gui**

Intérêt :

- Relecture personnelle
- Rédaction du commit log au fur et à mesure

⚠ **Misclick "Stage Hunk" / "Stage Lines"**

Retour d'expérience personnel : **git gui** patit d'un problème d'ergonomie gênant sur l'accès à la fonction "Stage Lines For Commit".

La fonction est accessible par clic droit uniquement dans le code, et le menu "Stage Lines For Commit" est situé juste en-dessous du menu "Stage Hunk For Commit".

De plus, le gestionnaire de souris pour l'application est un peu sensible.

Il arrive donc régulièrement qu'un click malencontreux sur "Stage Hunk For Commit" se fasse en lieu et place de "Stage Lines For Commit" comme attendu.

Cette erreur de manipulation a le fâcheux désagrément de faire tomber en *stage* toute une partie de code de façon inattendue. Généralement, cela nous coupe dans notre élan de commit ligne à ligne, ce qui est pénalisant pour l'exercice.

💡 **Solutions palliatives**

Options intéressantes pour contourner ce problème :

- **GitExtensions**

D'avis personnel, la fenêtre de commit de GitExtensions est une des meilleures que j'ai eu l'occasion d'utiliser.

Cette interface propose des raccourcis clavier 'S' (pour **stage*) et 'U' (pour *unstage*) très efficaces pour faire du commit ligne à ligne, et sans risque de misclick avec une fonction "Stage Hunk".

Elle permet également d'éditer le fichier source directement dans la fenêtre de commit, ce qui est fort pratique dans ce contexte de relecture pouvant amener à des corrections.

Il semble cependant que la procédure d'installation Linux tire beaucoup de contraintes (cf. https://git-extensions-documentation.readthedocs.io/en/release-2.51/getting_started.html#installation-linux-2-5x-only). Donc pas très cross-platform, essentiellement envisageable sur Windows.

Enfin, si la fenêtre de commit était pratique, GitExtensions avait globalement tendance à planter régulièrement.

- **GitKraken Desktop**

Dans sa fenêtre de commit, GitKraken Desktop propose des boutons **+** à gauche du code, sans fonction "Stage Hunk" à proximité, donc pas de risque de misclick.

Mais la bulle info a tendance à couvrir le code, ce qui gêne la lecture et pénalise l'exercice.

Il reste le click droit sur les lignes de code, sans fonction "Stage Hunk" à proximité encore une fois, mais moins pratique qu'un bouton directement accessible ou un raccourci clavier.

L'éditation de code dans la fenêtre de commit n'est pas disponible en mode *diff*, et nécessite de basculer en mode *file*, pour revenir ensuite en mode *diff* pour stager les lignes... Pas très pratique.

- **Patcher `git gui`**

Astuce inspirée de <https://stackoverflow.com/questions/32661397/is-there-a-keyboard-shortcut-for-stage-lines-in-git-gui#35543923>.

Comme indiqué dans le post *stackoverflow* cité, le principe est d'ajouter un raccourci clavier à `git gui` directement dans le code.

En effet, `git gui` est implémenté en `Tcl/Tk`, un langage de script permettant de réaliser des interfaces homme-machine. Et qui dit langage de script dit qu'on peut modifier le script !

Repérer le script `Tcl/Tk` de `git gui` :

- Sous Windows : `C:/Program Files (x86)/Git/libexec/git-core/git-gui.tcl` (d'après le post *stackoverflow*)
- Sous Linux : `/usr/lib/git-core/git-gui` (sur ma machine Ubuntu)

Editer le script, et ajouter les lignes suivantes en fin de fichier :

```
# Inspired from https://stackoverflow.com/questions/32661397/is-there-a-keyboard-shortcut-for-stage-lines-in-git-gui#35543923

bind .    <Key-F4> stagelines

proc stagelines {} {
    apply_or_revert_range_or_line %X %Y 0
}
```

```
# for older versions of git-gui, use this line instead:  
#apply_range_or_line %X %Y  
do_rescan  
}
```

Avec ce patch, la touche F4 devient un raccourci pour les fonctions "Stage Lines For Commit" et "Unstage Lines From Commit", ce qui permet d'éviter les risques de misclicks.

En revanche, pas de possibilité d'édition du fichier dans la fenêtre de commit. Il faut garder son éditeur à portée de main pour les corrections de dernière minute.

Merges & rebases

Merge

`git merge`

Principe du merge :

- Merges automatiques
- Merges conflictuels

Rebase

`git rebase`

Principe du rebase :

- Déplacement des noeuds
- Réécriture d'historique

Rebases conflictuels :

- `git status`
- `git rebase --continue`

Cherry-pick

`git cherry-pick`

Utilisation de frontends

- Tortoise Git
- Git Extension